



Events

Adding interactivity

What is an Event

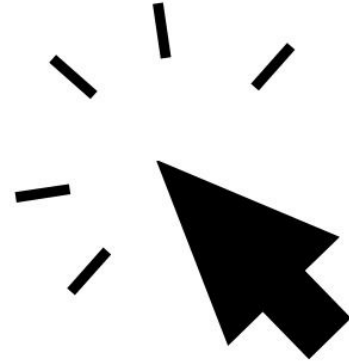
Events

- Things that happen on a webpage in our browser
- Our browser “fires” some kind of signal
- JavaScript can react to these signals
- Creates interactivity on the page



Some examples

- The user selects, clicks, or hovers the cursor over a certain element
- The user presses a key on the keyboard
- The user resizes or closes the browser window
- A web page finishes loading
- A form is submitted
- A video is played, paused, or ends
- An error occurs



Reacting to an event

Listen and handle

In Chrome, open your console on any website.

Paste the following snippet to see all events:

```
monitorEvents(window);
```

Note: this is a Chrome DevTools helper: it won't work in your JS files.



Listen and handle

"If a tree falls in a forest and no one is around to hear it, does it make a sound?" George Berkeley

- Events occur all the time...
- To react to an event, you need to attach an event listener to it
 - a block of code that runs when the event fires
 - sometimes also called event handlers



addEventListener

- [MDN](#)
- Available on all DOM elements
- Requires 2 parameters:
 - **type**: event type you want to listen to
 - **listener**: which function (= event handler) should be executed

```
<button>Click</button>
```

```
const $button = document.querySelector(`button`);  
$button.addEventListener(`click`, () => console.log(`click`));
```



addEventListener

// Anonymous arrow function

```
$button.addEventListener(`click`, () => console.log(`clicked`));
```

// Anonymous function with event object and block body

```
$button.addEventListener(`click`, event => {  
  console.log(`clicked`);  
  console.log(event.type);  
});
```

// Named function (preferred option)

```
$button.addEventListener(`click`, handleClickButton);
```



addEventListener

WARNING: no parentheses

Otherwise, function is executed immediately!

// Correct

```
$button.addEventListener(`click`, handleClickButton);
```

// Wrong

```
$button.addEventListener(`click`, handleClickButton());
```



addEventListener

Naming convention: handle[Event][Element]

- handleClickButton
- handleSubmitForm
- handleFocusInput
- handleBlurInput
- ...



removeEventListener

- [MDN](#)
- Same 2 parameters
 - **type**: event type you want to listen to
 - **listener**: which function (= event handler) should be stopped

```
const $button = document.querySelector(`button`);  
$button.removeEventListener(`click`, handleClickButton);
```



Types of events

- Overview on [MDN](#)
- The name of the event mostly tells you what it does



Types of events: common event types

click	the element clicked
dblclick	the element double-clicked
mouseenter	the mouse point enters to the element
mouseleave	the mouse pointer leaves the element
mousemove	the mouse pointer moves on the element
mouseover	the mouse pointer moves in on the element
mouseout	the mouse pointer moves out from the element
keyup	a key is up
keydown	a key is down
blur	the element is not focused
focus	the element is focused
input	value enter to input field
change	value change on input field



The Event Object

The Event Object

- Object passed as an argument with each event handler
- Contains additional information about the event depending on the type of event.
- For example:
 - mouse event: position of the mouse
 - keyboard event: which key was pressed

```
const handleClickButton = (event) => {  
  console.log(event);  
};
```



currentTarget vs target

currentTarget vs target

target = the DOM element that has triggered the event

currentTarget = the DOM element linked to the eventlistener

```
<ul> <li>Item</li> </ul>
```

```
$ul.addEventListener(`click`, (e) => {  
  console.log(e.target);           // <li>  
  console.log(e.currentTarget);    // <ul>  
});
```



preventDefault

preventDefault

- [MDN](#)
- Prevents the default behaviour of an event.
- We will handle the behaviour ourselves
- For example:
 - Stop navigation when clicking on a link to prevent complete page refresh
 - Stop submission of a form for validation

```
$form.addEventListener(`submit`, (event) => {  
  event.preventDefault();  
  // validate and handle manually  
});
```



Event Propagation

Event propagation

- [MDN](#)
- Events propagate to all parent elements!
- Clicking a child element = clicking all parent elements



stopPropagation

- [MDN](#)
- Stop the propagation of the event to parent elements (Bubbling)
- Prevents that the same event handler is executed multiple times

